

Deep Learning for NLP

Student name: *Ioannis Konstantinou*
sdi: *sdi1700067*

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *Spring Semester 2025*

Contents

1	Abstract	2
2	Data processing and analysis	2
2.1	Pre-processing	2
2.2	Data partitioning for train, test and validation	2
2.3	Vectorization	3
3	Algorithms and Experiments	3
3.1	Experiments	3
3.2	Hyper-parameter tuning	5
3.3	Optimization techniques	6
3.4	Evaluation	7
4	Results and Overall Analysis	7
4.1	Results Analysis	7
4.1.1	Best trial	8
4.2	Comparison with the first project	8

1. Abstract

Η εργασία αυτή αφορά την επίλυση ενός **Classification** προβλήματος. Με την χρήση ενός νευρωνικού δικτύου (**Feedforward Neural Network**) για την ταξινόμηση κειμένων σε δύο κατηγορίες. Η προσέγγιση περιλαμβάνει την προεπεξεργασία των δεδομένων, μετατροπή των κειμένων σε διανυσματικές αναπαραστάσεις χρησιμοποιώντας προ-εκπαιδευμένα **embeddings** (**GloVe** και του προ-εκπαιδευμένου **embeddings glove.twitter.27B.200d**) και εκπαίδευση του νευρωνικού δικτύου. Για την βελτιστοποίηση των υπερπαραμέτρων του μοντέλου έγινε με τη χρήση του **Optuna framework**, με στόχο τη μεγιστοποίηση της ακρίβειας στην επικύρωση (**validation accuracy**). Στην υλοποίησή μου στο **Kaggle** έχω σε σχόλια το **optuna** μιας και παίρνει αρκετό χρόνο και είπατε ότι μπορούμε να το έχουμε σε σχόλια για να μην το τρέχουμε εμείς.

2. Data processing and analysis

2.1. Pre-processing

Τα βήματα της προεπεξεργασίας των δεδομένων είναι κυρίως από την προηγούμενη εργασία και βασίστηκα πάνω σε αυτά κάνοντας μερικές μικρές αλλαγές. Γενικά τα βήματα της προεπεξεργασίας των δεδομένων περιλαμβάνουν:

- Μετατροπή όλου του κειμένου σε πεζά (**lowercasing**) για ομογενοποίηση.
- Κανονικοποίηση των κενών διαστημάτων και αφαίρεση κενών στην αρχή και το τέλος των συμβολοσειρών.
- Αφαίρεση **URLs** και αναφορών σε εικόνες ('http', 'www', 'pic').
- Αντικατάσταση συντομογραφιών και **slang** λέξεων που βρίσκονται συχνά σε κείμενα **social media**/ιντερνετ (π.χ., 'n't' σε 'not', 'r' σε 'are', 'u' σε 'you', 'lol', 'brb', κ.λπ.) με τις ολόκληρες λέξεις. Αυτό βοηθά στην ομαλή αντιστοίχιση με τα **embeddings**.
- Αφαίρεση ειδικών χαρακτήρων, συμβόλων και αναφορών χρηστών (διάφορα σημεία στίξης/σύμβολα).
- Δημιουργία **tokens** (λέξεων) από το καθαρισμένο κείμενο με χρήση του **word tokenize**.
- Εφαρμογή **Lemmatization** με χρήση του '**nlk.stem.WordNetLemmatizer**' για τη μείωση των λέξεων στην αρχική τους μορφή (π.χ., "running" → "run"), μειώνοντας έτσι το μέγεθος του λεξιλογίου και βοηθώντας στη γενίκευση (στην εργασία στο **kaggle** το έβαλα σε σχόλια γιατί δεν μπορούσα να το κατεβάσω, αλλά γενικά δεν είχε καμία αλλαγή στο **accuracy score** που είχε σε **unseen data**).

Ο κώδικας δεν περιλαμβάνει αφαίρεση πολύ μικρών προτάσεων ή **stopwords**, επειδή κατά την χρήση τους ερίχναν την απόδοση του μοντέλου αισθητά, οπότε και τα έβαλα και δεν φαίνονται στην εργασία.

2.2. Data partitioning for train, test and validation

Ο παρεχόμενος κώδικας φορτώνει τα δεδομένα από τρία ξεχωριστά αρχεία **CSV** τα οποία είναι δοσμένα ήδη από το **competition**: **train dataset.csv**, **val dataset.csv**, και **test dataset.csv**. Σε γενικές γραμμές, η χρήση των **dataset** γίνεται για τους εξής λόγους:

- **Σύνολο Εκπαίδευσης (Train Set):** Χρησιμοποιείται για την εκπαίδευση του μοντέλου, δηλαδή για την εκμάθηση των βαρών του δικτύου.
- **Σύνολο Επικύρωσης (Validation Set):** Χρησιμοποιείται κατά τη διάρκεια της εκπαίδευσης για την παρακολούθηση της απόδοσης του μοντέλου σε δεδομένα που δεν έχει "δει" κατά την εκπαίδευση. Είναι κρίσιμο για τη βελτιστοποίηση των υπερπαραμέτρων και την εφαρμογή τεχνικών όπως το **early stopping**, βοηθώντας στην αποφυγή του **overfitting**.
- **Σύνολο Ελέγχου (Test Set):** Χρησιμοποιείται ΜΟΝΟ ΜΙΑ ΦΟΡΑ στο τέλος, αφού έχει ολοκληρωθεί η εκπαίδευση και η βελτιστοποίηση των υπερπαραμέτρων, για μια αμερόληπτη εκτίμηση της τελικής απόδοσης του μοντέλου σε άγνωστα δεδομένα.

2.3. Vectorization

Για τη μετατροπή των προεπεξεργασμένων κειμένων σε αριθμητικές αναπαραστάσεις που μπορούν να χρησιμοποιηθούν από το νευρωνικό δίκτυο χρησιμοποιήσα προ-εκπαιδευμένα **word embeddings** από το GloVe.

- **Μέθοδος:** Χρησιμοποιούνται τα προ-εκπαιδευμένα **embeddings GloVe (Global Vectors for Word Representation)**, συγκεκριμένα το μοντέλο **glove.twitter.27B.200d.txt**, το οποίο έχει εκπαιδευτεί σε 27 δισεκατομμύρια **tokens** από Twitter και έχει διάσταση 200 διαστάσεων ανά λέξη. Έγινε και χρήση του **GoogleNews-vectors-negative300.bin** αλλά τα αποτελέσματα στο **accuracy** ήταν ελαφρώς χειρότερα από εκείνα του **glove twitter 27B 200d** και για αυτό προτιμήθηκε αυτό.
- **Διαδικασία:**
 1. Το αρχείο GloVe μετατρέπεται σε μορφή συμβατή με **word2vec (glv-with-w2v-format.txt)** χρησιμοποιώντας το **gensim.scripts.glove2word2vec**.
 2. Το μοντέλο GloVe φορτώνεται σε μορφή **KeyedVectors** από το Gensim (**gensim.models.keyedvectors.KeyedVectors**).
 3. Εάν μια πρόταση δεν περιέχει κανένα **token** που να υπάρχει στο λεξιλόγιο του GloVe (π.χ., είναι κενή μετά την προεπεξεργασία ή περιέχει μόνο άγνωστες λέξεις), η συνάρτηση επιστρέφει ένα διάνυσμα μηδενικών της αντίστοιχης διάστασης (200).
- **Scaling:** Αφού υπολογιστούν τα διανύσματα για όλες τις προτάσεις, εφαρμόζεται τυπική κλιμάκωση (**Standard Scaling**) στα δεδομένα με χρήση του **StandardScaler** από το **sklearn.preprocessing**. Αυτό μετατρέπει τα χαρακτηριστικά ώστε να έχουν μέσο όρο 0 και τυπική απόκλιση 1, κάτι που μπορεί να βοηθήσει στην καλύτερη και ταχύτερη σύγκλιση του νευρωνικού δικτύου. Σημειώνεται ότι ο **scaler** εκπαιδεύεται (**fit**) μόνο στα δεδομένα εκπαίδευσης και στη συνέχεια εφαρμόζεται (**transform**) τόσο στα δεδομένα εκπαίδευσης όσο και στα δεδομένα επικύρωσης (και ελέγχου, αν χρησιμοποιηθούν).

3. Algorithms and Experiments

3.1. Experiments

Η προσέγγιση ξεκίνησε με την επιλογή μιας βασικής αρχιτεκτονικής **FNN**, παρόμοιας με αυτές που εξερευνήθηκαν κατά τη διάρκεια των εργαστηρίων. Αυτό το αρχικό **"baseline"**

μοντέλο πέτυχε ακρίβεια στο σύνολο εκπαίδευσης 0.7540 και στο σύνολο επικύρωσης 0.7502. Αυτό το αποτέλεσμα αποτέλεσε το σημείο εκκίνησης για περαιτέρω πειραματισμούς.

Στη συνέχεια, δοκιμάστηκαν διάφορες τεχνικές προεπεξεργασίας δεδομένων και αρχιτεκτονικές βελτιώσεις για να διερευνηθεί η επίδρασή τους στην απόδοση. Τα πειράματα αυτά περιλάμβαναν:

- **Negation Handling:** Ενσωματώθηκε χειρισμός άρνησης στην προεπεξεργασία των κειμένων. Ωστόσο, αυτό οδήγησε σε μια μικρή μείωση της ακρίβειας τόσο στο σύνολο εκπαίδευσης (0.7520) όσο και στο σύνολο επικύρωσης (0.7494). Δεδομένου ότι δεν υπήρξε βελτίωση, αυτή η τεχνική δεν διατηρήθηκε.
- Αφαίρεση επιπλέον κενών διαστημάτων: Πραγματοποιήθηκε καθαρισμός των κειμένων αφαιρώντας τα περιττά κενά. Αυτή η απλή τεχνική είχε θετική επίδραση, αυξάνοντας την ακρίβεια σε **train: 0.7630** και **validation: 0.7552**. Η βελτίωση αποδίδεται στην καλύτερη κανονικοποίηση των δεδομένων εισόδου.
- Αφαίρεση ειδικών χαρακτήρων: Σε συνδυασμό με την αφαίρεση των επιπλέον κενών, αφαιρέθηκαν και ειδικοί χαρακτήρες από τα κείμενα. Αυτό επέφερε περαιτέρω αύξηση της ακρίβειας σε **train: 0.7632** και **validation: 0.7626**, υποδηλώνοντας ότι οι ειδικοί χαρακτήρες δεν προσέθεταν χρήσιμη πληροφορία για την ταξινόμηση και η αφαίρεσή τους μείωσε το "θόρυβο".
- Ενσωμάτωση **Dropout**: Προστέθηκε η τεχνική **Dropout** στην αρχιτεκτονική του δικτύου με μια αρχική τιμή ποσοστού. Αυτό είχε ως αποτέλεσμα την αύξηση της ακρίβειας σε **train: 0.7770** και **validation: 0.7692**, υποδεικνύοντας την ικανότητα του **Dropout** να βοηθήσει στην καταπολέμηση του **overfitting**, επιτρέποντας στο μοντέλο να γενικεύει καλύτερα.
- Αριθμός επιπέδων: Κατά την πορεία των πειραμάτων και ειδικά κατά την αντιμετώπιση φαινομένων **overfitting**, διερευνήθηκε η επίδραση του αριθμού των κρυφών επιπέδων. Από ένα αρχικό μοντέλο που μπορεί να περιλάμβανε περισσότερα επίπεδα, επιλέχθηκε μια αρχιτεκτονική με δύο κρυφά επίπεδα ('block1', 'block2' στον κώδικα) ως ένα καλό σημείο ισορροπίας μεταξύ χωρητικότητας μοντέλου και κινδύνου **overfitting**. Η βελτιστοποίηση των μεγεθών αυτών των δύο επιπέδων (H1, H2) έγινε στη συνέχεια με το **Optuna**.

Με χρήση του **Optuna** επικεντρώθηκαν στην επίδραση των εξής υπερπαραμέτρων:

- Αριθμός νευρώνων στα κρυφά επίπεδα H1 και H2. Δοκιμάστηκαν διαφορετικοί συνδυασμοί μεγεθών για να βρεθεί η βέλτιστη χωρητικότητα του μοντέλου. Κυρίως μεταξύ 2, 3 και 4 κρυφών επιπέδων.
- Συναρτήσεις Ενεργοποίησης: Δοκιμάστηκαν οι LeakyReLU, ReLU και SELU για να διαπιστωθεί ποια αποδίδει καλύτερα σε αυτό το πρόβλημα.
- **Optimizer**: Συγκρίθηκαν οι Adam, AdamW, SGD και RMSprop.
- **Learning Rate**: Βελτιστοποιήθηκε η αρχική τιμή του ρυθμού μάθησης.
- **Dropout Rate**: Βελτιστοποιήθηκε το ποσοστό των νευρώνων που αγνοούνται τυχαία κατά την εκπαίδευση για την καταπολέμηση του **overfitting**. Στην αρχή με 0.1 έως 0.5 αλλά λόγω μεγάλου **overfit** το ανέβασα στα 0.4 έως 0.5 .

- **Weight Decay:** Βελτιστοποιήθηκε ο συντελεστής για την L2 regularization στα βάρη του δικτύου.
- **Batch Size:** Δοκιμάστηκαν μεγέθη batch 32, 64 και 128.

Κάθε trial του Optuna περιλάμβανε τη χρήση ενός learning rate scheduler (ReduceLROnPlateau) και early stopping με υπομονή (patience), βασισμένο στην απώλεια επικύρωσης (validation loss), για την περαιτέρω αποφυγή του overfitting και την επιτάχυνση της διαδικασίας.

Η απόδοση κάθε πειράματος (trial) μετρήθηκε με βάση την ακρίβεια στο σύνολο επικύρωσης (validation accuracy). Το Optuna στόχευσε στη μεγιστοποίηση αυτής της ακρίβειας (με ελαχιστοποίηση του 1 - accuracy).

3.2. Hyper-parameter tuning

Η βελτιστοποίηση των υπερπαραμέτρων πραγματοποιήθηκε συστηματικά χρησιμοποιώντας το Optuna. Ο στόχος ήταν η ελαχιστοποίηση του (1 - Validation Accuracy), που είναι ισοδύναμο με τη μεγιστοποίηση της ακρίβειας στο σύνολο επικύρωσης. Το Optuna εξερεύνησε το χώρο υπερπαραμέτρων που ορίστηκε στην συνάρτηση 'objective'.

Η επιλογή των υπερπαραμέτρων έγινε από τους εξής πιθανούς χώρους:

- optimizer name: ["Adam", "AdamW"]
- learning rate: Log-uniform μεταξύ 10^{-4} και 10^{-2}
- dropout rate: Uniform μεταξύ 0.4 και 0.5 (με βήμα 0.05)
- weight decay: Log-uniform μεταξύ 10^{-3} και 10^{-1}
- activation: ["LeakyReLU", "ReLU", "SELU"]
- batch size: [32, 64, 128]
- H1: Ακέραιες τιμές μεταξύ 32 και 128 (με βήμα 32)
- H2: Ακέραιες τιμές μεταξύ 16 και 96 (με βήμα 16)

Για κάθε συνδυασμό υπερπαραμέτρων που πρότεινε το Optuna, το μοντέλο εκπαιδευόταν για διάφορες εποχές (max epochs), αλλά λόγω του overfitting έγινε η επιλογή στις 50 (δοκιμάστηκαν και 100 αλλά το πρόβλημα του overfitting γινόταν ακόμα πιο μεγάλο). Για την αντιμετώπιση του overfitting και την επιτάχυνση της εκπαίδευσης, χρησιμοποιήθηκαν:

- **Early Stopping:** Η εκπαίδευση σταματούσε πρόωρα εάν η απώλεια στο σύνολο επικύρωσης δεν βελτιωνόταν για 10 συνεχόμενες εποχές (patience=10). Αυτό διασφαλίζει ότι δεν εκπαιδεύουμε το μοντέλο υπερβολικά, με κίνδυνο να αρχίσει να απομνημονεύει τα δεδομένα εκπαίδευσης αντί να μαθαίνει γενικά χαρακτηριστικά.
- **ReduceLROnPlateau Scheduler:** Ο ρυθμός μάθησης μειωνόταν κατά 50% εάν η απώλεια στο σύνολο επικύρωσης δεν βελτιωνόταν για 3 συνεχόμενες εποχές (patience=3, factor=0.5). Αυτό βοηθά το μοντέλο να κάνει πιο λεπτές ρυθμίσεις στα βάρη όταν η βελτίωση επιβραδύνεται.

- **Dropout:** Το ποσοστό Dropout (ρυθμίστηκε μεταξύ 0.4 και 0.5) απενεργοποιεί τυχαία νευρώνες κατά την εκπαίδευση, αναγκάζοντας το δίκτυο να βασίζεται σε διαφορετικά υποσύνολα νευρώνων, καθιστώντας το λιγότερο ευαίσθητο σε συγκεκριμένα χαρακτηριστικά και πιο ανθεκτικό στο **overfitting**.
- **Weight Decay :** Προσθέτει μια ποινή στην απώλεια ανάλογη με το τετράγωνο των βαρών, αποθαρρύνοντας τα μεγάλα βάρη και μειώνοντας την πολυπλοκότητα του μοντέλου, κάτι που επίσης βοηθά στην καταπολέμηση του **overfitting**.
- **Batch Normalization:** Εφαρμόζεται μετά από κάθε γραμμικό επίπεδο, σταθεροποιώντας τις κατανομές των **activations** και επιτρέποντας τη χρήση υψηλότερων ρυθμών μάθησης, ενώ λειτουργεί και ως μορφή **regularization**.

Κατά την εξέλιξη των πειραμάτων με το Optuna, παρατηρήθηκε αρχικά ότι η αύξηση των εποχών εκπαίδευσης από 5 σε 10 βελτίωσε **accuracy score** (π.χ., από 0.7692 σε 0.7749), υποδηλώνοντας ότι το μοντέλο χρειαζόταν περισσότερο χρόνο για να μάθει. Ωστόσο, περαιτέρω αύξηση των εποχών (π.χ., σε ένα πείραμα με 100 εποχές) οδήγησε σε εμφάνιση έντονου **overfitting**, όπου το **accuracy** συνέχιζε να αυξάνεται σημαντικά (π.χ., 0.8467), ενώ το **accuracy** είτε σταθεροποιούνταν είτε έπεφτε ελαφρά (π.χ., 0.7700), διευρύνοντας σημαντικά τη διαφορά μεταξύ τους.

Η συστηματική βελτιστοποίηση μέσω Optuna, σε συνδυασμό με την ενσωμάτωση **Early Stopping** και του **ReduceLROnPlateau scheduler** εντός κάθε **trial**, αντιμετώπισε αυτό το πρόβλημα. Το **Early Stopping** εξασφάλισε ότι η εκπαίδευση σταματούσε μόλις το **accuracy** σταθεροποιούνταν ή άρχιζε να αυξάνεται, περιορίζοντας το **overfitting**. Ο **scheduler** βοηθούσε το μοντέλο να κάνει πιο λεπτές ρυθμίσεις αργότερα στην εκπαίδευση. Επιπλέον, οι τεχνικές **Dropout** και **Weight Decay**, των οποίων οι συντελεστές βελτιστοποιήθηκαν από το Optuna, συνέβαλαν ενεργά στη μείωση της πολυπλοκότητας του μοντέλου και την αύξηση της ανθεκτικότητάς του στο **overfitting**.

Το καλύτερο **trial** που βρέθηκε από το Optuna πέτυχε την υψηλότερη ακρίβεια στο **accuracy** (περίπου 0.7869) με αντίστοιχο **accuracy** στο σύνολο εκπαίδευσης περίπου 0.8024. Υπάρχει ένα μικρό κενό μεταξύ των δύο ακριβειών, υποδεικνύοντας την παρουσία ενός μικρού βαθμού **overfitting**. Ωστόσο, αυτή η διαμόρφωση επιλέχθηκε ως βέλτιστη επειδή παρουσίασε την καλύτερη επίδοση στο **accuracy**.

3.3. Optimization techniques

Οι τεχνικές βελτιστοποίησης που χρησιμοποιήθηκαν για την εκπαίδευση του νευρωνικού δικτύου είναι οι εξής:

- **Optimizer:** Χρησιμοποιήθηκαν οι αλγόριθμοι βελτιστοποίησης Adam, AdamW. Η επιλογή μεταξύ των δύο ήταν μέρος της βελτιστοποίησης υπερπαραμέτρων με το Optuna.
- **Loss Function:** Χρησιμοποιήθηκε η συνάρτηση απώλειας **nn.BCEWithLogitsLoss** από την PyTorch. Αυτή είναι μια συνάρτηση που συνδυάζει τη **Sigmoid activation function** και την **Binary Cross Entropy (BCE) loss**. Είναι κατάλληλη για δυαδικά προβλήματα ταξινόμησης και είναι πιο αριθμητικά σταθερή από τη χρήση **Sigmoid** και **BCE** ξεχωριστά.
- **Learning Rate Scheduling:** Εφαρμόστηκε ο **Scheduler ReduceLROnPlateau**. Αυτός ο scheduler παρακολουθεί μια μετρική την **validation loss** και μειώνει το **learning rate**

όταν η βελτίωση της μετρικής σταματά (**plateaus**). Αυτό βοηθά το **optimizer** να συγκλίνει πιο ομαλά προς ένα τοπικό ή καθολικό ελάχιστο της συνάρτησης απώλειας.

- **Regularization:** Όπως αναφέρθηκε και στην ενότητα βελτιστοποίησης υπερπαραμέτρων, χρησιμοποιήθηκαν τεχνικές **regularization** όπως **Dropout** και **Weight Decay** για να αποφευχθεί το **overfitting** και να βελτιωθεί η γενίκευση του μοντέλου.
- **Batch Normalization:** Εφαρμόστηκε μετά από κάθε γραμμικό επίπεδο στο μοντέλο, σταθεροποιώντας τις κατανομές των **activations** και επιτρέποντας τη χρήση υψηλότερων ρυθμών μάθησης, αλλά σε αυτή την εργασία λειτούργησε ως μορφή **regularization**.

3.4. Evaluation

Ο κώδικας χρησιμοποιεί την ακρίβεια (**Accuracy**) ως την κύρια μετρική για τη βελτιστοποίηση υπερπαραμέτρων μέσω του **Optuna** και η τελική αξιολόγηση του μοντέλου γίνεται στο **test set**, το οποίο δεν έχει χρησιμοποιηθεί καθόλου κατά την εκπαίδευση ή τη βελτιστοποίηση υπερπαραμέτρων.

4. Results and Overall Analysis

4.1. Results Analysis

Βάσει των αποτελεσμάτων του **Optuna** η βελτιστοποίηση των υπερπαραμέτρων οδήγησε σε μια βελτίωση της επίδοσης σε σύγκριση με το αρχικό **baseline**.

Η ανάλυση των αποτελεσμάτων του **Optuna** δείχνει ότι η ακρίβεια στο **accuracy** ήταν κοντά στο 78% . Η καλύτερη επίδοση επιτεύχθηκε με **Validation Accuracy 0.7821**. Είναι καλύτερο από την επίδοση του αρχικού **baseline** μοντέλου **0.7502**.

Μεταξύ των **accuracy** του **train** και του **validation**, υπάρχει ένα μικρό κενό (περίπου 1.04%). Αυτό υποδηλώνει την παρουσία ενός μικρού βαθμού **overfitting**. Αυτό έχει επιτευχθεί λόγω του **regularization** (**Dropout**, **Weight Decay**, **Batch Normalization**) και τους μηχανισμούς **Early Stopping** και **Learning Rate Scheduling** που χρησιμοποιήθηκαν κατά την εκπαίδευση. Αυτοί οι μηχανισμοί λειτούργησαν αποτελεσματικά για να σταματήσουν την εκπαίδευση πριν το **overfitting** γίνει ανεξέλεγκτο και να βοηθήσουν το μοντέλο να βρει μια διαμόρφωση που γενικεύει καλά στα **validation** δεδομένα.

Από την ανάλυση των υπερπαραμέτρων των **trials**, φαίνεται ότι διάφοροι συνδυασμοί μπορούν να οδηγήσουν σε παρόμοιες, σχετικά υψηλές ακρίβειες. Για παράδειγμα, το πείραμα που πέτυχε **validation Accuracy: 0.7805** είχε διαφορετικά μεγέθη κρυφών επιπέδων (**H1=96**, **H2=32**) και **activation function** (**LeakyReLU**) σε σύγκριση με το βέλτιστο (**H1=32**, **H2=96**, **Activation=LeakyReLU**), υποδεικνύοντας ότι υπήρχε κάποια ευελιξία στις αρχιτεκτονικές επιλογές εντός του χώρου αναζήτησης που ορίστηκε. Ο **Optimizer 'AdamW'** εμφανίστηκε στο βέλτιστο **trial**, μαζί με το **'Adam'**, δείχνοντας ότι και οι δύο ήταν βιώσιμες επιλογές, με το **AdamW** να δίνει την τελικά καλύτερη επίδοση σε αυτή τη συγκεκριμένη αναζήτηση. Οι τιμές για **Learning Rate**, **Dropout** και **Weight Decay** βρέθηκαν εντός των λογικών περιοχών που ορίστηκαν, συμβάλλοντας στην σταθερότητα και την επίδοση του μοντέλου. Η χρήση **Batch Size 128** φάνηκε στο βέλτιστο **trial**.

Περαιτέρω πειράματα που θα μπορούσαν να δοκιμαστούν περιλαμβάνουν:

- Χρήση διαφορετικών προ-εκπαιδευμένων **embeddings** ή **embeddings** μεγαλύτερης διάστασης.

- Δοκιμή διαφορετικών μεθόδων για τη δημιουργία αναπαράστασης πρότασης από **word embeddings**.
- Δοκιμή διαφορετικών αρχιτεκτονικών δικτύων (π.χ. προσθήκη περισσότερων κρυφών επιπέδων).
- Πιο εκτενής βελτιστοποίηση υπερπαραμέτρων με **Optuna** (π.χ. περισσότερα **trials**).
- Προσθήκη επιπλέον βημάτων προεπεξεργασίας.

4.1.1. Best trial. Το καλύτερο **trial**, όπως προσδιορίστηκε από την εκτέλεση του **Optuna study** (με βάση τη μέγιστη **Validation Accuracy**), είχε τις εξής υπερπαραμέτρους:

- **Optimizer:** AdamW
- **Learning Rate:** 0.0041696733 (περίπου 0.00417)
- **Dropout Rate:** 0.45
- **Weight Decay:** 0.0010285156 (περίπου 0.00103)
- **Activation Function:** LeakyReLU
- **Hidden Layer 1 Size (H1):** 32
- **Hidden Layer 2 Size (H2):** 96
- **Batch Size:** 128
- **Validation Accuracy:** 0.78207850 (περίπου 0.7821)

4.2. Comparison with the first project

Σε σύγκριση με την πρώτη εργασία, το **accuracy score** ήταν εμφανώς χειρότερο.

- **Word Embeddings:** Τα προ-εκπαιδευμένα **embeddings** συλλαμβάνουν σημασιολογικές και συντακτικές σχέσεις μεταξύ λέξεων, κάτι που οι απλές μετρήσεις συχνότητας (όπως το TF-IDF) δεν μπορούν να κάνουν.
- **Neural Networks:** Τα νευρωνικά δίκτυα μπορούν να μάθουν πιο πολύπλοκες, μη-γραμμικές σχέσεις στα δεδομένα σε σύγκριση με γραμμικά μοντέλα.
- **End-to-end Learning (μετά τα embeddings):** Το δίκτυο εκπαιδεύεται για να βρει τα βέλτιστα βάρη που αντιστοιχίζουν τα διανύσματα των προτάσεων στον στόχο ταξινόμησης.

Κατα την γνώμη μου το χειρότερο **accuracy score** είναι εξετίας της ανεπαρκούς βελτιστοποίησης των υπερπαραμέτρων του νευρωνικού δικτύου, ή η επιλογή αρχιτεκτονικής/**embeddings** που δεν ταιριάζει καλά με τη φύση των δεδομένων.